

TCGA Differential Gene Expression Computation

Bokang Shi

Visiting Student, Imperial College London

Cell Systems Laboratory, Institute for Protein Research, Osaka University

Summer Internship

Supervisor: Prof. Mariko Okada

Date: 9 July 2026

Data

- TCGA RNA-seq
- TCGAbiolinks
- GDC STAR Counts workflow

Workflow

1. Query TCGA project with TCGAbiolinks.
2. Download tumour and normal samples.
3. Skip projects with fewer than two tumour or two normal samples.
4. Prepare count matrix.
5. Filter low-expression genes.
6. Perform differential expression analysis using DESeq2 with the experimental design defined by sample condition (tumour vs normal).
7. Export DEG results as CSV.

Output

Each cancer type produces one CSV containing:

- gene ID
- gene symbol
- log2FoldChange
- p-value
- adjusted p-value

Current limitation

Only TCGA normal samples are currently used.

GTEx integration and batch-effect correction have not yet been implemented.

R script

C:\Users\Bokan\Documents\context-dependent-GRNs\scripts\compute_tcga_deseq2.R

```
0001 #!/usr/bin/env Rscript
0002
0003 # TCGA tumor-vs-normal DESeq2 result tables.
0004
0005 suppressPackageStartupMessages({
0006   library(DESeq2)
0007   library(TCGAbiolinks)
0008   library(SummarizedExperiment)
0009 })
0010
0011 parse_args <- function(args) {
0012   out <- list(
0013     projects_csv = "outputs/tcga_degs/manifest/tcga_projects.csv",
0014     output_dir = "outputs/tcga_degs",
0015     gdc_data_dir = "data/gdc",
0016     projects = "",
0017     max_projects = NA_integer_,
0018     workflow_type = "STAR - Counts",
0019     tumor_sample_types = "Primary Tumor",
0020     normal_sample_types = "Solid Tissue Normal",
0021     # Placeholder threshold for the current TCGA-only run; review before final analysis.
0022     min_normal = 2L,
0023     min_tumor = 2L,
0024     min_count = 10L,
0025     min_count_samples = 3L,
0026     dry_run = FALSE,
0027     resume = FALSE
0028   )
0029
0030   i <- 1L
0031   while (i <= length(args)) {
0032     key <- args[[i]]
0033     if (key == "--dry-run") {
0034       out$dry_run <- TRUE
0035       i <- i + 1L
0036       next
0037     }
0038     if (key == "--resume") {
0039       out$resume <- TRUE
0040       i <- i + 1L
0041       next
0042     }
0043     if (i == length(args)) {
0044       stop(sprintf("Missing value for argument %s", key))
0045     }
0046     value <- args[[i + 1L]]
0047     if (key == "--projects-csv") out$projects_csv <- value
0048     else if (key == "--output-dir") out$output_dir <- value
0049     else if (key == "--gdc-data-dir") out$gdc_data_dir <- value
0050     else if (key == "--projects") out$projects <- value
0051     else if (key == "--max-projects") out$max_projects <- as.integer(value)
0052     else if (key == "--workflow-type") out$workflow_type <- value
0053     else if (key == "--tumor-sample-types") out$tumor_sample_types <- value
0054     else if (key == "--normal-sample-types") out$normal_sample_types <- value
0055     else if (key == "--min-normal") out$min_normal <- as.integer(value)
0056     else if (key == "--min-tumor") out$min_tumor <- as.integer(value)
0057     else if (key == "--min-count") out$min_count <- as.integer(value)
0058     else if (key == "--min-count-samples") out$min_count_samples <- as.integer(value)
0059     else stop(sprintf("Unknown argument: %s", key))
0060     i <- i + 2L
0061   }
0062   out
0063 }
0064
0065 first_existing_col <- function(df, candidates) {
0066   for (candidate in candidates) {
0067     if (candidate %in% colnames(df)) return(candidate)
0068   }
0069   NA_character_
0070 }
0071
0072 split_config_values <- function(value, default) {
0073   if (is.null(value) || length(value) == 0L || is.na(value) || !nzchar(trimws(value))) {
0074     value <- default
0075   }
0076   parts <- unlist(strsplit(as.character(value), "\\s*;\\s*|\\s*\\\\\\s*"))
0077   parts <- trimws(parts)
0078   parts[nzchar(parts)]
0079 }
```

```

0079 }
0080
0081 collapse_values <- function(values) {
0082   values <- unique(values[!is.na(values) & nzchar(values)])
0083   if (length(values) == 0L) return("")
0084   paste(values, collapse = "; ")
0085 }
0086
0087 manifest_value <- function(project_row, column, default) {
0088   if (column %in% names(project_row)) {
0089     value <- project_row[[column]]
0090     if (!is.na(value) && nzchar(trimws(as.character(value)))) return(as.character(value))
0091   }
0092   default
0093 }
0094
0095 project_config <- function(project_row, cfg) {
0096   workflow <- manifest_value(project_row, "workflow_type", cfg$workflow_type)
0097   tumor_source <- manifest_value(project_row, "tumor_source", "GDC")
0098   normal_source <- manifest_value(project_row, "normal_source", "GDC")
0099   list(
0100     workflow_type = workflow,
0101     tumor_source = tumor_source,
0102     normal_source = normal_source,
0103     tumor_sample_types = split_config_values(
0104       manifest_value(project_row, "tumor_sample_types", cfg$tumor_sample_types),
0105       cfg$tumor_sample_types
0106     ),
0107     normal_sample_types = split_config_values(
0108       manifest_value(project_row, "normal_sample_types", cfg$normal_sample_types),
0109       cfg$normal_sample_types
0110     )
0111   )
0112 }
0113
0114 select_count_assay <- function(se) {
0115   available <- SummarizedExperiment::assayNames(se)
0116   preferred <- c("unstranded", "stranded_first", "stranded_second")
0117   selected <- preferred[preferred %in% available]
0118   if (length(selected) > 0L) return(selected[[1L]])
0119   available[[1L]]
0120 }
0121
0122 make_gene_annotation <- function(se) {
0123   row_data <- as.data.frame(SummarizedExperiment::rowData(se))
0124   gene_id_col <- first_existing_col(row_data, c("gene_id", "ensembl_gene_id", "ensembl_id"))
0125   gene_symbol_col <- first_existing_col(row_data, c("gene_name", "external_gene_name", "gene_symbol"))
0126   gene_type_col <- first_existing_col(row_data, c("gene_type", "gene_biotype", "external_gene_type"))
0127
0128   gene_id <- rownames(se)
0129   if (!is.na(gene_id_col)) gene_id <- as.character(row_data[[gene_id_col]])
0130
0131   gene_symbol <- rep(NA_character_, nrow(row_data))
0132   if (!is.na(gene_symbol_col)) gene_symbol <- as.character(row_data[[gene_symbol_col]])
0133
0134   gene_type <- rep(NA_character_, nrow(row_data))
0135   if (!is.na(gene_type_col)) gene_type <- as.character(row_data[[gene_type_col]])
0136
0137   data.frame(
0138     gene_id = gene_id,
0139     gene_symbol = gene_symbol,
0140     gene_type = gene_type,
0141     stringsAsFactors = FALSE
0142   )
0143 }
0144
0145 discover_gdc_metadata <- function(project_id) {
0146   query <- GDCQuery(
0147     project = project_id,
0148     data.category = "Transcriptome Profiling",
0149     data.type = "Gene Expression Quantification"
0150   )
0151   getResults(query)
0152 }
0153
0154 metadata_profile <- function(metadata) {
0155   if (is.null(metadata) || nrow(metadata) == 0L) {
0156     return(list(
0157       sample_type_col = NA_character_,
0158       workflow_col = NA_character_,
0159       sample_types_found = character(),
0160       workflow_types_found = character()
0161     ))
0162   }
0163   sample_type_col <- first_existing_col(metadata, c("sample_type", "cases.samples.sample_type"))

```

```

0164 workflow_col <- first_existing_col(metadata, c("analysis_workflow_type", "workflow_type"))
0165 sample_types <- character()
0166 workflow_types <- character()
0167 if (!is.na(sample_type_col)) sample_types <- sort(unique(as.character(metadata[[sample_type_col]])))
0168 if (!is.na(workflow_col)) workflow_types <- sort(unique(as.character(metadata[[workflow_col]])))
0169 list(
0170   sample_type_col = sample_type_col,
0171   workflow_col = workflow_col,
0172   sample_types_found = sample_types,
0173   workflow_types_found = workflow_types
0174 )
0175 }
0176
0177 diagnose_metadata <- function(metadata, profile, cfg_row) {
0178   if (is.null(metadata) || nrow(metadata) == 0L) {
0179     return("no_gdc_expression_quantification_files")
0180   }
0181   if (is.na(profile$sample_type_col)) return("sample_type_column_not_found")
0182   if (is.na(profile$workflow_col)) return("workflow_type_column_not_found")
0183   if (!cfg_row$workflow_type %in% profile$workflow_types_found) {
0184     return(sprintf("requested_workflow_unavailable: %s", cfg_row$workflow_type))
0185   }
0186   missing_tumor <- setdiff(cfg_row$tumor_sample_types, profile$sample_types_found)
0187   if (length(missing_tumor) > 0L) {
0188     return(sprintf("requested_tumor_sample_type_unavailable: %s", collapse_values(missing_tumor)))
0189   }
0190   if (toupper(cfg_row$normal_source) != "GDC") {
0191     return(sprintf("external_normal_source_not_supported_in_this_script: %s", cfg_row$normal_source))
0192   }
0193   missing_normal <- setdiff(cfg_row$normal_sample_types, profile$sample_types_found)
0194   if (length(missing_normal) > 0L) {
0195     return(sprintf("requested_normal_sample_type_unavailable: %s", collapse_values(missing_normal)))
0196   }
0197   ""
0198 }
0199
0200 filter_metadata_for_config <- function(metadata, profile, cfg_row) {
0201   if (is.null(metadata) || nrow(metadata) == 0L ||
0202     is.na(profile$sample_type_col) || is.na(profile$workflow_col)) {
0203     return(metadata[0, , drop = FALSE])
0204   }
0205   sample_types <- unique(c(cfg_row$tumor_sample_types, cfg_row$normal_sample_types))
0206   metadata[
0207     metadata[[profile$workflow_col]] == cfg_row$workflow_type &
0208     metadata[[profile$sample_type_col]] %in% sample_types,
0209     ,
0210     drop = FALSE
0211   ]
0212 }
0213
0214 count_samples <- function(metadata, profile, cfg_row) {
0215   if (is.null(metadata) || nrow(metadata) == 0L || is.na(profile$sample_type_col)) {
0216     return(list(n_tumor = 0L, n_normal = 0L))
0217   }
0218   list(
0219     n_tumor = sum(metadata[[profile$sample_type_col]] %in% cfg_row$tumor_sample_types, na.rm = TRUE),
0220     n_normal = sum(metadata[[profile$sample_type_col]] %in% cfg_row$normal_sample_types, na.rm = TRUE)
0221   )
0222 }
0223
0224 make_gdc_query <- function(project_id, cfg_row) {
0225   GDCQuery(
0226     project = project_id,
0227     data.category = "Transcriptome Profiling",
0228     data.type = "Gene Expression Quantification",
0229     workflow.type = cfg_row$workflow_type,
0230     sample.type = unique(c(cfg_row$tumor_sample_types, cfg_row$normal_sample_types))
0231   )
0232 }
0233
0234 summary_fields <- c(
0235   "tcga_abbreviation", "project_id", "run_mode", "status",
0236   "tumor_source", "normal_source",
0237   "configured_tumor_sample_types", "configured_normal_sample_types",
0238   "selected_workflow_type", "sample_types_found", "workflow_types_found",
0239   "n_tumor", "n_normal", "n_genes_tested", "output_file",
0240   "skip_reason", "message"
0241 )
0242
0243 write_empty_summary <- function(path) {
0244   empty <- as.data.frame(setNames(replicate(length(summary_fields), character()), simplify = FALSE),
0245     summary_fields)
0246   write.csv(empty, path, row.names = FALSE)
0247 }

```

```

0248 read_resume_summary <- function(path) {
0249   if (!file.exists(path)) {
0250     return(as.data.frame(setNames(replicate(length(summary_fields), character()), simplify = FALSE),
summary_fields)))
0251   }
0252   existing <- read.csv(path, stringsAsFactors = FALSE, check.names = FALSE)
0253   for (field in summary_fields) {
0254     if (!field %in% names(existing)) existing[[field]] <- NA_character_
0255   }
0256   existing[, summary_fields, drop = FALSE]
0257 }
0258
0259 find_resume_row <- function(existing_summary, project_row) {
0260   if (nrow(existing_summary) == 0L) return(NULL)
0261   matches <- existing_summary[
0262     existing_summary$tcga_abbreviation == project_row$tcga_abbreviation[[1L]] &
0263     existing_summary$project_id == project_row$project_id[[1L]],
0264     ,
0265     drop = FALSE
0266   ]
0267   if (nrow(matches) == 0L) return(NULL)
0268   row <- matches[nrow(matches), , drop = FALSE]
0269   reusable_status <- row$status %in% c("completed", "skipped", "dry_run_ready")
0270   output_ok <- !identical(row$status[[1L]], "completed") ||
0271     (nzchar(row$output_file[[1L]]) && file.exists(row$output_file[[1L]]))
0272   if (isTRUE(reusable_status) && isTRUE(output_ok)) row else NULL
0273 }
0274
0275 summary_row_df <- function(result) {
0276   normalized <- lapply(summary_fields, function(field) {
0277     value <- result[[field]]
0278     if (is.null(value) || length(value) == 0L) return(NA)
0279     value[[1L]]
0280   })
0281   names(normalized) <- summary_fields
0282   as.data.frame(normalized, stringsAsFactors = FALSE)
0283 }
0284
0285 base_result <- function(tcga_abbreviation, project_id, cfg_row, profile, cfg) {
0286   list(
0287     tcga_abbreviation = tcga_abbreviation,
0288     project_id = project_id,
0289     run_mode = ifelse(isTRUE(cfg$dry_run), "dry_run", "full"),
0290     status = "",
0291     tumor_source = cfg_row$tumor_source,
0292     normal_source = cfg_row$normal_source,
0293     configured_tumor_sample_types = collapse_values(cfg_row$tumor_sample_types),
0294     configured_normal_sample_types = collapse_values(cfg_row$normal_sample_types),
0295     selected_workflow_type = cfg_row$workflow_type,
0296     sample_types_found = collapse_values(profile$sample_types_found),
0297     workflow_types_found = collapse_values(profile$workflow_types_found),
0298     n_tumor = NA_integer_,
0299     n_normal = NA_integer_,
0300     n_genes_tested = NA_integer_,
0301     output_file = "",
0302     skip_reason = "",
0303     message = ""
0304   )
0305 }
0306
0307 compute_project <- function(project_row, cfg) {
0308   tcga_abbreviation <- project_row$tcga_abbreviation[[1L]]
0309   project_id <- project_row$project_id[[1L]]
0310   cfg_row <- project_config(project_row, cfg)
0311
0312   message(sprintf("[%s] Discovering GDC metadata", project_id))
0313   metadata <- discover_gdc_metadata(project_id)
0314   profile <- metadata_profile(metadata)
0315   result <- base_result(tcga_abbreviation, project_id, cfg_row, profile, cfg)
0316
0317   selected_metadata <- filter_metadata_for_config(metadata, profile, cfg_row)
0318   counts <- count_samples(selected_metadata, profile, cfg_row)
0319   result$n_tumor <- counts$n_tumor
0320   result$n_normal <- counts$n_normal
0321
0322   diagnostic <- diagnose_metadata(metadata, profile, cfg_row)
0323   if (nzchar(diagnostic)) {
0324     result$status <- "skipped"
0325     result$skip_reason <- diagnostic
0326     result$message <- "Configured workflow/sample types are not available in discovered GDC metadata"
0327     return(result)
0328   }
0329
0330   if (counts$n_tumor < cfg$min_tumor) {
0331     result$status <- "skipped"

```

```

0332     result$skip_reason <- sprintf("insufficient_tumor_samples: n_tumor=%s, min_tumor=%s", counts$n_tumor,
0333     cfg$min_tumor)
0333     result$message <- "DESeq2 was not run"
0334     return(result)
0335   }
0336   if (counts$n_normal < cfg$min_normal) {
0337     result$status <- "skipped"
0338     result$skip_reason <- sprintf("no_sufficient_tcga_normal_samples: n_normal=%s, min_normal=%s",
0339     counts$n_normal, cfg$min_normal)
0339     result$message <- "DESeq2 was not run"
0340     return(result)
0341   }
0342
0343   if (cfg$dry_run) {
0344     result$status <- "dry_run_ready"
0345     result$message <- sprintf("Ready for DESeq2 download/run with %s files", nrow(selected_metadata))
0346     return(result)
0347   }
0348
0349   query <- make_gdc_query(project_id, cfg_row)
0350   GDCdownload(query, method = "api", files.per.chunk = 20, directory = cfg$gdc_data_dir)
0351   se <- GDCprepare(query, directory = cfg$gdc_data_dir)
0352
0353   col_data <- as.data.frame(SummarizedExperiment::colData(se))
0354   sample_type_col <- first_existing_col(col_data, c("sample_type", "definition"))
0355   if (is.na(sample_type_col)) {
0356     stop("Could not find sample type column in prepared GDC object")
0357   }
0358
0359   requested_sample_types <- unique(c(cfg_row$tumor_sample_types, cfg_row$normal_sample_types))
0360   keep_samples <- col_data[[sample_type_col]] %in% requested_sample_types
0361   se <- se[, keep_samples]
0362   col_data <- as.data.frame(SummarizedExperiment::colData(se))
0363   condition <- ifelse(col_data[[sample_type_col]] %in% cfg_row$normal_sample_types, "normal", "tumor")
0364   condition <- factor(condition, levels = c("normal", "tumor"))
0365
0366   n_tumor <- sum(condition == "tumor")
0367   n_normal <- sum(condition == "normal")
0368   if (n_tumor < cfg$min_tumor || n_normal < cfg$min_normal) {
0369     stop(sprintf("Insufficient samples after preparation: n_tumor=%s, n_normal=%s", n_tumor, n_normal))
0370   }
0371
0372   assay_name <- select_count_assay(se)
0373   counts_matrix <- SummarizedExperiment::assay(se, assay_name)
0374   storage.mode(counts_matrix) <- "integer"
0375
0376   sample_df <- data.frame(condition = condition, row.names = colnames(counts_matrix))
0377   dds <- DESeqDataSetFromMatrix(
0378     countData = counts_matrix,
0379     colData = sample_df,
0380     design = ~ condition
0381   )
0382
0383   keep_genes <- rowSums(DESeq2::counts(dds) >= cfg$min_count) >= cfg$min_count_samples
0384   dds <- dds[keep_genes, ]
0385   dds <- DESeq(dds)
0386
0387   res <- results(dds, contrast = c("condition", "tumor", "normal"))
0388   res_df <- as.data.frame(res)
0389   annotation <- make_gene_annotation(se)[keep_genes, , drop = FALSE]
0390   out_df <- cbind(annotation, res_df)
0391
0392   out_df$tcga_abbreviation <- tcga_abbreviation
0393   out_df$project_id <- project_id
0394   out_df$n_tumor <- n_tumor
0395   out_df$n_normal <- n_normal
0396
0397   ordered_cols <- c(
0398     "tcga_abbreviation", "project_id", "gene_id", "gene_symbol", "gene_type",
0399     "baseMean", "log2FoldChange", "lfcSE", "stat", "pvalue", "padj",
0400     "n_tumor", "n_normal"
0401   )
0402   out_df <- out_df[, ordered_cols]
0403
0404   project_out_dir <- file.path(cfg$output_dir, "deseq2_results", tcga_abbreviation)
0405   dir.create(project_out_dir, recursive = TRUE, showWarnings = FALSE)
0406   output_file <- file.path(project_out_dir, "deseq2_results.csv")
0407   write.csv(out_df, output_file, row.names = FALSE)
0408
0409   result$status <- "completed"
0410   result$n_tumor <- n_tumor
0411   result$n_normal <- n_normal
0412   result$n_genes_tested <- nrow(out_df)
0413   result$output_file <- normalizePath(output_file, winslash = "\\", mustWork = FALSE)
0414   result$message <- sprintf("Assay=%s", assay_name)

```

```

0415     result
0416 }
0417
0418 main <- function() {
0419     cfg <- parse_args(commandArgs(trailingOnly = TRUE))
0420     dir.create(file.path(cfg$output_dir, "manifest"), recursive = TRUE, showWarnings = FALSE)
0421     dir.create(cfg$gdc_data_dir, recursive = TRUE, showWarnings = FALSE)
0422
0423     projects <- read.csv(cfg$projects_csv, stringsAsFactors = FALSE, check.names = FALSE)
0424     required <- c("tcga_abbreviation", "project_id")
0425     missing <- setdiff(required, colnames(projects))
0426     if (length(missing) > 0L) {
0427         stop(sprintf("Project manifest is missing columns: %s", paste(missing, collapse = ", ")))
0428     }
0429
0430     if (nzchar(cfg$projects)) {
0431         selected <- trimws(strsplit(cfg$projects, ",")[[1L]])
0432         projects <- projects[projects$tcga_abbreviation %in% selected | projects$project_id %in% selected, ]
0433     }
0434     if (!is.na(cfg$max_projects)) {
0435         projects <- head(projects, cfg$max_projects)
0436     }
0437     if (nrow(projects) == 0L) {
0438         stop("No projects selected")
0439     }
0440
0441     summary_path <- file.path(cfg$output_dir, "manifest", "tcga_deg_summary.csv")
0442     existing_summary <- read_resume_summary(summary_path)
0443     if (!cfg$resume) {
0444         write_empty_summary(summary_path)
0445         existing_summary <- read_resume_summary(summary_path)
0446     }
0447     summary_rows <- list()
0448
0449     for (i in seq_len(nrow(projects))) {
0450         project_row <- projects[i, , drop = FALSE]
0451         project_id <- project_row$project_id[[1L]]
0452         resume_row <- if (cfg$resume) find_resume_row(existing_summary, project_row) else NULL
0453         if (!is.null(resume_row)) {
0454             message(sprintf("[%s] resume: reusing existing %s row", project_id, resume_row$status[[1L]]))
0455             summary_rows[[length(summary_rows) + 1L]] <- resume_row
0456             summary_df <- do.call(rbind, summary_rows)
0457             write.csv(summary_df, summary_path, row.names = FALSE)
0458             next
0459         }
0460         result <- tryCatch(
0461             compute_project(project_row, cfg),
0462             error = function(e) {
0463                 cfg_row <- project_config(project_row, cfg)
0464                 profile <- list(sample_types_found = character(), workflow_types_found = character())
0465                 out <- base_result(project_row$tcga_abbreviation[[1L]], project_id, cfg_row, profile, cfg)
0466                 out$status <- "error"
0467                 out$skip_reason <- "runtime_error"
0468                 out$message <- conditionMessage(e)
0469                 out
0470             }
0471         )
0472         summary_rows[[length(summary_rows) + 1L]] <- summary_row_df(result)
0473         summary_df <- do.call(rbind, summary_rows)
0474         write.csv(summary_df, summary_path, row.names = FALSE)
0475         message(sprintf("[%s] %s: %s", project_id, result$status, result$message))
0476     }
0477
0478     message(sprintf("Wrote summary: %s", normalizePath(summary_path, winslash = "\\ ", mustWork = FALSE)))
0479 }
0480
0481 main()

```